Five Bluetooth Nodes — One Component per Arduino UNO + HC-05

Oct 15, 2026 · @Vishal

All code for this guide is in the GitHub repository VishalRamKN/arduino-bluetooth-nodes, under the nodes folder and the HC-05 renaming tool. Clone it with `git clone https://github.com/VishalRamKN/arduino-bluetooth-nodes.git`, or use **Code → Download ZIP** on GitHub, then open the sketch for your node:

Node	Sketch in the repo
HC-05 renaming tool	`tools/HC05_AT_Mode/HC05_AT_Mode.ino`
1 LCD	`nodes/01-lcd/BT_LCD/BT_LCD.ino`
2 Ultrasonic	`nodes/02-ultrasonic/BT_DIST/BT_DIST.ino`
3 Servo	`nodes/03-servo/BT_SERVO/BT_SERVO.ino`
4 Heartbeat	`nodes/04-heartbeat/BT_HEART/BT_HEART.ino`
5 LED	`nodes/05-led/BT_LED/BT_LED.ino`

The code listings in this guide are the same files, kept here for reading alongside the explanations. The repo's SETUP.md has the step-by-step student setup.

Overview

The hub is split into five independent nodes: each node is its own Arduino UNO + HC-05 with exactly one component, its own sketch, and its own Bluetooth name. The phone connects to whichever node it wants to control.

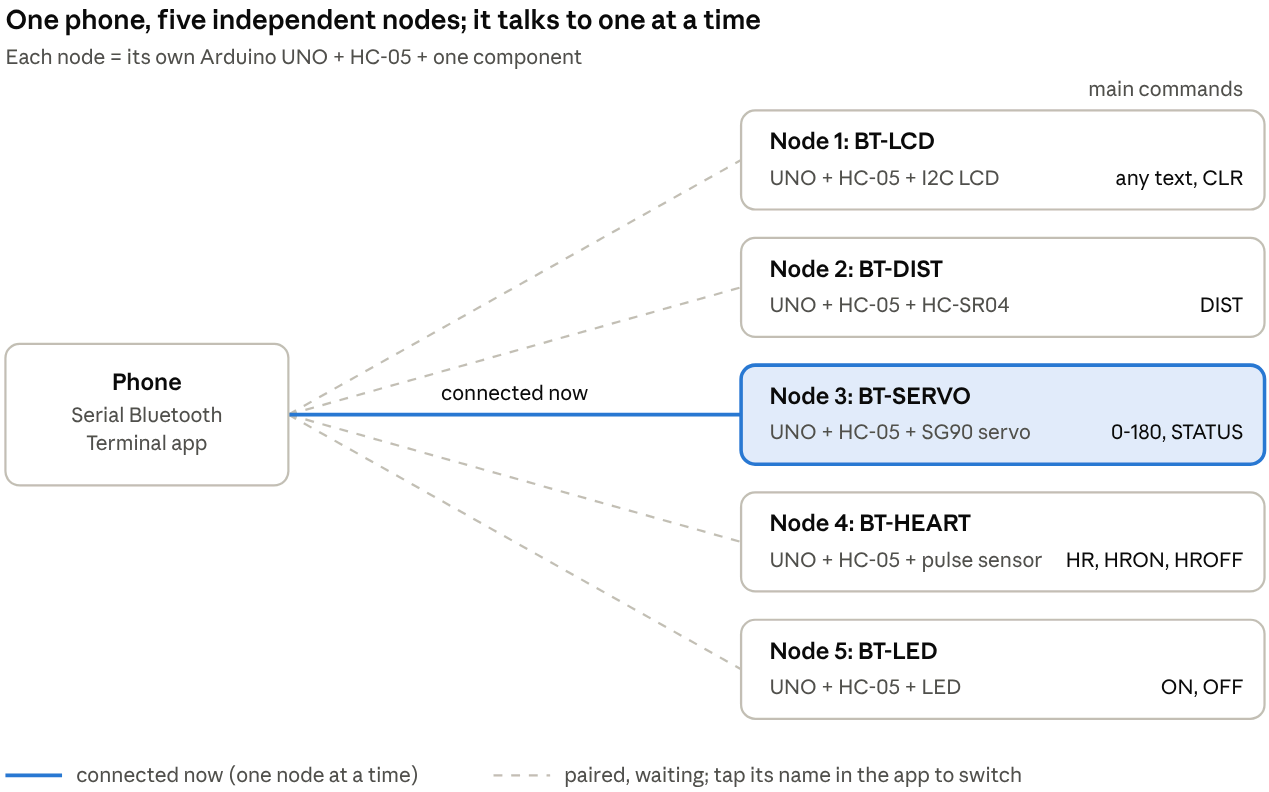
Why split it: a fault in one node (a servo browning out, a loose sensor wire) cannot affect the others, each sketch is short and easy to debug, and nodes can sit in different places.

Every node uses the same base kit (UNO, HC-05, 1 kΩ + 2 kΩ divider) and the same command style, so learning one node teaches you all five.

Node	Component	Bluetooth name	Extra parts on top of the base kit	Main commands
1	I2C LCD 16x2	`BT-LCD`	None	any text, `CLR`
2	HC-SR04 ultrasonic	`BT-DIST`	None	`DIST`
3	SG90 servo	`BT-SERVO`	470 µF capacitor; optional external 5 V supply	`90`, `SERVO:90`, `STATUS`
4	Pulse (heartbeat) sensor	`BT-HEART`	None	`HR`, `HRON`, `HROFF`
5	LED	`BT-LED`	220 Ω resistor	`ON`, `OFF`

All five nodes also answer `HELP` (list of commands), `STATUS` (current state) and `PING` (replies with the node name, to confirm which node you are talking to).

System diagram



system diagram · one phone, five nodes

The nodes never talk to each other; the phone is the only link, and it holds one connection at a time. In this example it is connected to the servo node while the other four wait.

Base kit: UNO + HC-05 (same on every node)

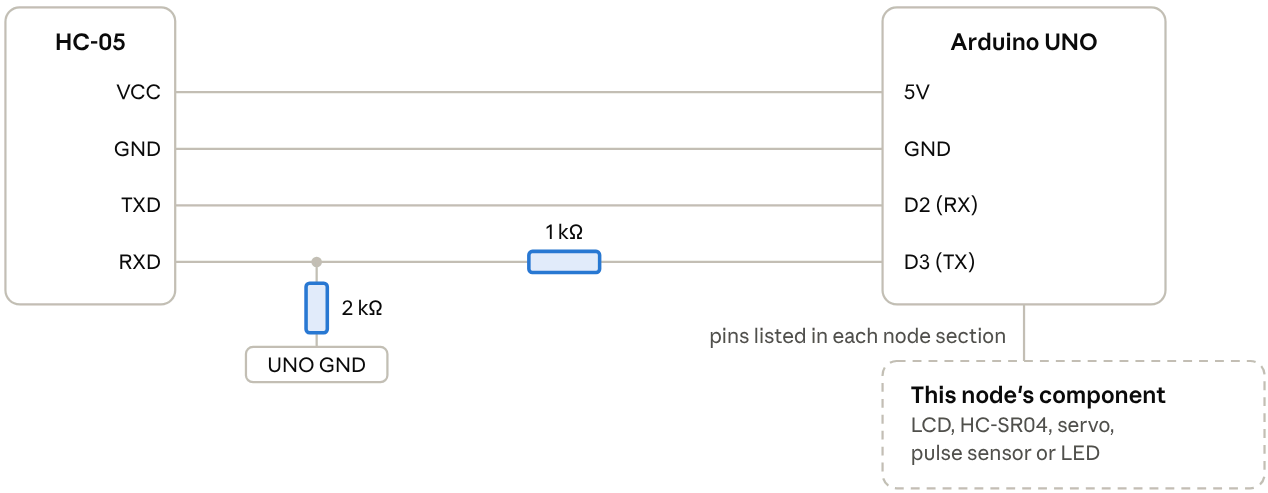
Build this five times; it is identical on every node, and only the component wired next to it changes.

HC-05 pin	UNO pin	Notes
VCC	5V	The HC-05 board has its own 3.3 V regulator
GND	GND	
TXD	D2	SoftwareSerial RX, direct wire
RXD	D3	Through a divider: D3 → 1 kΩ → RXD, and RXD → 2 kΩ → GND

HC-05 pin	UNO pin	Notes
STATE	Not connected	
EN / KEY	Not connected	Used only once, to enter AT mode for renaming

Same base wiring on all five nodes; only HC-05 RXD needs a divider

Resistors in blue form the 5 V to 3.3 V divider



base kit wiring · UNO + HC-05 with divider

The divider matters because HC-05 RXD is a 3.3 V input and D3 outputs 5 V; 5 × 2 / (1 + 2) ≈ 3.3 V. HC-05 TXD outputs 3.3 V, which the UNO reads as HIGH without help.

Using D2/D3 instead of D0/D1 keeps USB free, so you can upload sketches with the HC-05 still connected and also type commands in the Serial Monitor (9600 baud, "Both NL & CR") for testing.

Each node needs its own power: a USB cable, a 9 V adapter on the barrel jack, or a 7–12 V battery pack on VIN.

Naming each HC-05

All HC-05 modules ship named "HC-05", so five of them look identical on the phone; give each one a unique name once, before loading its node sketch.

Upload the pass-through sketch first (in the repo at tools/HC05_AT_Mode/HC05_AT_Mode.ino). It forwards what you type in the Serial Monitor to the HC-05 at 38400 baud, the AT-mode speed:

```
// HC-05 AT-mode pass-through (use once per module, for renaming)
#include <SoftwareSerial.h>
SoftwareSerial bt(2, 3);          // D2 <- TXD, D3 -> RXD (through divider)

void setup() {
  Serial.begin(9600);             // Serial Monitor speed
  bt.begin(38400);                // HC-05 AT-mode speed
  Serial.println("AT mode ready. Type AT and press Enter.");
}

void loop() {
  if (bt.available())     Serial.write(bt.read());
  if (Serial.available()) bt.write(Serial.read());
}
```

1. Upload the sketch with the HC-05 wired as in the base kit.
2. Unplug the HC-05's VCC wire. Press and hold the small button on the HC-05, plug VCC back in, then release. The LED now blinks slowly (about once every 2 s), which means AT mode.
3. Open Serial Monitor at 9600 baud with "Both NL & CR".
4. Type `AT` → it should answer `OK`.
5. Type the node's name, for example `AT+NAME=BT-LCD` → `OK`. Some newer firmware wants quotes: `AT+NAME="BT-LCD"`.
6. Check the data speed with `AT+UART?`. It should say `9600,0,0`; if not, send `AT+UART=9600,0,0`.
7. Optional: change the pairing PIN with `AT+PSWD=4321` (or `AT+PSWD="4321"`).
8. Unplug and re-plug the HC-05 without the button; the LED blinks fast again (normal mode). Now upload that node's sketch.

Repeat for each module: `BT-LCD`, `BT-DIST`, `BT-SERVO`, `BT-HEART`, `BT-LED`. A small label on each board saves confusion later.

Node 1 — LCD (BT-LCD)

Anything you type that is not a command appears on the LCD, so on this node you just type your message and press Send.

Pins

LCD pin	UNO pin
GND	GND
VCC	5V
SDA	A4
SCL	A5

Commands

Send	What happens	Reply
`Hello World` (any text)	Shown on LCD: characters 1–16 on line 1, 17–32 on line 2	`LCD: shown`
`LCD:help`	Shows text that would otherwise be a command word	`LCD: shown`
`CLR`	Clears the screen	`LCD: cleared`
`STATUS`	Reports what is on screen	`LCD shows: Hello World`
`PING` / `HELP`	Node name / command list	`BT-LCD here`

Code

Needs the **LiquidCrystal I2C** library by Frank de Brabander (Library Manager).

In the repo: `nodes/01-lcd/BT_LCD/BT_LCD.ino`, with a node README in the same `nodes/01-lcd` folder.

```
/* Node 1 - BT-LCD : Arduino UNO + HC-05 + I2C LCD 16x2 */
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include <SoftwareSerial.h>
#include <string.h>

const char NODE_NAME[] = "BT-LCD";
```

```
const uint8_t LCD_ADDR = 0x27;      // try 0x3F if the screen stays blank

SoftwareSerial bt(2, 3);            // D2 <- HC-05 TXD, D3 -> HC-05 RXD
(divider)
LiquidCrystal_I2C lcd(LCD_ADDR, 16, 2);

const uint8_t CMD_MAX = 64;
char btBuf[CMD_MAX + 1];  uint8_t btLen = 0;
char usbBuf[CMD_MAX + 1]; uint8_t usbLen = 0;
char shown[33] = "";                // what is on the screen now

// ---- replies go to phone and USB ----
void reply(const char *m)               { bt.println(m); Serial.println(m);
}
void reply(const __FlashStringHelper *m) { bt.println(m); Serial.println(m);
}

char *trim(char *s) {
  while (*s == ' ') s++;
  size_t n = strlen(s);
  while (n > 0 && s[n - 1] == ' ') s[--n] = '\0';
  return s;
}

// ---- LCD ----
void showOnLcd(const char *text) {
  lcd.clear();
  lcd.setCursor(0, 0);
  size_t n = strlen(text);
  for (size_t i = 0; i < n && i < 32; i++) {
    if (i == 16) lcd.setCursor(0, 1);   // wrap to line 2
    lcd.print(text[i]);
  }
  strncpy(shown, text, 32);
  shown[32] = '\0';
}

void printText(const char *text) {
  showOnLcd(text);
  if (strlen(text) > 32) reply(F("LCD: shown (cut to 32 chars)"));
  else reply(F("LCD: shown"));
}
```

```
// ---- commands ----
void handleCommand(char *cmd) {
  cmd = trim(cmd);
  if (*cmd == '\0') return;
  char out[48];

  if (strcasecmp(cmd, "PING") == 0) {
    snprintf(out, sizeof(out), "%s here", NODE_NAME);
    reply(out);
  } else if (strcasecmp(cmd, "CLR") == 0) {
    lcd.clear();
    shown[0] = '\0';
    reply(F("LCD: cleared"));
  } else if (strcasecmp(cmd, "STATUS") == 0) {
    if (shown[0] == '\0') reply(F("LCD is blank"));
    else { snprintf(out, sizeof(out), "LCD shows: %s", shown); reply(out); }
  } else if (strcasecmp(cmd, "HELP") == 0) {
    reply(F("BT-LCD commands:"));
    reply(F("<any text>   show on LCD"));
    reply(F("LCD:<text>   show text, even a command word"));
    reply(F("CLR          clear screen"));
    reply(F("STATUS       what is shown"));
    reply(F("PING         node name"));
  } else if (strncasecmp(cmd, "LCD:", 4) == 0) {
    printText(trim(cmd + 4));
  } else {
    printText(cmd);                    // plain text goes straight to the
LCD
  }
}

void pollStream(Stream &s, char *buf, uint8_t &len) {
  while (s.available()) {
    char c = s.read();
    if (c == '\n' || c == '\r') {
      if (len > 0) { buf[len] = '\0'; handleCommand(buf); len = 0; }
    } else if (len < CMD_MAX) {
      buf[len++] = c;
    }
  }
}

void setup() {
```

```
  Serial.begin(9600);
  bt.begin(9600);
  lcd.init();
  lcd.backlight();
  showOnLcd("BT-LCD ready");
  reply(F("BT-LCD ready. Send HELP."));
}

void loop() {
  pollStream(bt, btBuf, btLen);          // from phone
  pollStream(Serial, usbBuf, usbLen);    // from Serial Monitor
}
```

How it works

- `pollStream()` collects characters until Enter, then calls `handleCommand()`. It runs for both Bluetooth and USB.
- `handleCommand()` checks the reserved words first (`PING`, `CLR`, `STATUS`, `HELP`, `LCD:`); everything else is treated as text for the screen.
- `showOnLcd()` prints up to 32 characters across two lines and keeps a copy in `shown`, which `STATUS` reports.

Test

Send `PING` → `BT-LCD here`. Send `Hello World` → it appears on the LCD. Send `CLR` → screen clears.

Node 2 — Ultrasonic (BT-DIST)

Send `DIST` and the node takes three quick readings and replies with the middle one in centimetres, which filters out the odd bad echo.

Pins

HC-SR04 pin	UNO pin
VCC	5V
Trig	D7

HC-SR04 pin	UNO pin
Echo	D6
GND	GND

Commands

Send	What happens	Reply
`DIST` (or `D`)	Measures distance	`Distance: 23.4 cm` or `Distance: out of range`
`STATUS`	Repeats the last reading and how long ago it was taken	`Last: 23.4 cm, 12 s ago`
`PING` / `HELP`	Node name / command list	`BT-DIST here`

Code

No extra libraries needed.

In the repo: `nodes/02-ultrasonic/BT_DIST/BT_DIST.ino`, with a node README in the same `nodes/02-ultrasonic` folder.

```
/* Node 2 - BT-DIST : Arduino UNO + HC-05 + HC-SR04 */
#include <SoftwareSerial.h>
#include <string.h>
#include <stdlib.h>

const char NODE_NAME[] = "BT-DIST";
const uint8_t TRIG_PIN = 7;
const uint8_t ECHO_PIN = 6;

SoftwareSerial bt(2, 3);            // D2 <- HC-05 TXD, D3 -> HC-05 RXD
(divider)

const uint8_t CMD_MAX = 64;
char btBuf[CMD_MAX + 1];  uint8_t btLen = 0;
char usbBuf[CMD_MAX + 1]; uint8_t usbLen = 0;
```

```
float lastCm = -1;                  // -1 = no reading yet / out of range
unsigned long lastAt = 0;

void reply(const char *m)               { bt.println(m); Serial.println(m);
}
void reply(const __FlashStringHelper *m) { bt.println(m); Serial.println(m);
}

char *trim(char *s) {
  while (*s == ' ') s++;
  size_t n = strlen(s);
  while (n > 0 && s[n - 1] == ' ') s[--n] = '\0';
  return s;
}

// ---- one raw reading ----
float pingOnce() {
  digitalWrite(TRIG_PIN, LOW);
  delayMicroseconds(2);
  digitalWrite(TRIG_PIN, HIGH);
  delayMicroseconds(10);              // 10 us trigger pulse
  digitalWrite(TRIG_PIN, LOW);
  unsigned long dur = pulseIn(ECHO_PIN, HIGH, 30000UL); // 30 ms = ~5 m
  if (dur == 0) return -1;
  return dur * 0.0343 / 2.0;          // cm, there and back
}

// ---- median of three readings ----
float readDistanceCm() {
  float a = pingOnce(); delay(60);    // let echoes die out between pings
  float b = pingOnce(); delay(60);
  float c = pingOnce();
  if (a > b) { float t = a; a = b; b = t; }
  if (b > c) { float t = b; b = c; c = t; }
  if (a > b) { float t = a; a = b; b = t; }
  return b;                           // middle value
}

void handleCommand(char *cmd) {
  cmd = trim(cmd);
  if (*cmd == '\0') return;
  char out[48], num[10];
```

```
  if (strcasecmp(cmd, "DIST") == 0 || strcasecmp(cmd, "D") == 0) {
    lastCm = readDistanceCm();
    lastAt = millis();
    if (lastCm < 0) reply(F("Distance: out of range"));
    else {
      dtostrf(lastCm, 0, 1, num);
      snprintf(out, sizeof(out), "Distance: %s cm", num);
      reply(out);
    }
  } else if (strcasecmp(cmd, "STATUS") == 0) {
    if (lastAt == 0) reply(F("No reading yet. Send DIST."));
    else {
      unsigned long ago = (millis() - lastAt) / 1000;
      if (lastCm < 0) snprintf(out, sizeof(out), "Last: out of range, %lu s
ago", ago);
      else { dtostrf(lastCm, 0, 1, num);
             snprintf(out, sizeof(out), "Last: %s cm, %lu s ago", num, ago);
}
      reply(out);
    }
  } else if (strcasecmp(cmd, "PING") == 0) {
    snprintf(out, sizeof(out), "%s here", NODE_NAME);
    reply(out);
  } else if (strcasecmp(cmd, "HELP") == 0) {
    reply(F("BT-DIST commands:"));
    reply(F("DIST or D   measure distance"));
    reply(F("STATUS      last reading"));
    reply(F("PING        node name"));
  } else {
    snprintf(out, sizeof(out), "Unknown: %s (send HELP)", cmd);
    reply(out);
  }
}

void pollStream(Stream &s, char *buf, uint8_t &len) {
  while (s.available()) {
    char c = s.read();
    if (c == '\n' || c == '\r') {
      if (len > 0) { buf[len] = '\0'; handleCommand(buf); len = 0; }
    } else if (len < CMD_MAX) {
      buf[len++] = c;
    }
  }
```

```
}

void setup() {
  Serial.begin(9600);
  bt.begin(9600);
  pinMode(TRIG_PIN, OUTPUT);
  pinMode(ECHO_PIN, INPUT);
  reply(F("BT-DIST ready. Send HELP."));
}

void loop() {
  pollStream(bt, btBuf, btLen);
  pollStream(Serial, usbBuf, usbLen);
}
```

How it works

- `pingOnce()` sends a 10 µs pulse on Trig. The sensor fires an ultrasonic burst and holds Echo HIGH until the sound returns; `pulseIn()` measures that time.
- Sound covers 0.0343 cm per µs and the time is for the round trip, so distance = time × 0.0343 / 2.
- `readDistanceCm()` takes three readings 60 ms apart and returns the middle one. A single stray echo cannot then spoil the result. The whole measurement takes about 0.2 s.
- No echo within 30 ms (beyond about 5 m) returns `out of range`. The HC-SR04 works best between 2 cm and 400 cm.

Test

Hold a book 20 cm from the sensor and send `DIST`; the reply should be close to 20 cm. Then send `STATUS`.

Node 3 — Servo (BT-SERVO)

Send an angle from 0 to 180 (just the number, or `SERVO:90`) and the servo turns there; `STATUS` tells you the angle it is at.

Pins

SG90 wire	UNO pin	Notes
Brown	GND	
Red	5V	Fine on this node since the servo is the only load; an external 5 V supply is steadier (join its GND to UNO GND)
Orange	D9	Control signal

Put a 470 µF capacitor across the servo's red and brown wires (stripe to brown). It absorbs the current spike when the servo starts moving, which otherwise can reset the UNO or drop the Bluetooth link.

Commands

Send	What happens	Reply
`90` (any 0–180)	Turns to that angle	`Servo moved to 90 deg`
`SERVO:45`	Same, long form	`Servo moved to 45 deg`
`STATUS` (or `SERVO?`)	Reports the current angle	`Servo is at 45 deg`
`PING` / `HELP`	Node name / command list	`BT-SERVO here`
`200` or `abc`	Rejected	`Error: send a number 0 to 180`

Code

Uses the built-in Servo library.

In the repo: `nodes/03-servo/BT_SERVO/BT_SERVO.ino`, with a node README in the same `nodes/03-servo` folder.

```
/* Node 3 - BT-SERVO : Arduino UNO + HC-05 + SG90 servo */
#include <SoftwareSerial.h>
#include <Servo.h>
#include <string.h>
#include <stdlib.h>
```

```
const char NODE_NAME[] = "BT-SERVO";
const uint8_t SERVO_PIN = 9;

SoftwareSerial bt(2, 3);            // D2 <- HC-05 TXD, D3 -> HC-05 RXD
(divider)
Servo servo;

const uint8_t CMD_MAX = 64;
char btBuf[CMD_MAX + 1];  uint8_t btLen = 0;
char usbBuf[CMD_MAX + 1]; uint8_t usbLen = 0;

int servoAngle = 90;                // start position

void reply(const char *m)               { bt.println(m); Serial.println(m);
}
void reply(const __FlashStringHelper *m) { bt.println(m); Serial.println(m);
}

char *trim(char *s) {
  while (*s == ' ') s++;
  size_t n = strlen(s);
  while (n > 0 && s[n - 1] == ' ') s[--n] = '\0';
  return s;
}

// 1 to 3 digits only
bool isNumber(const char *s) {
  if (*s == '\0' || strlen(s) > 3) return false;
  for (; *s; s++) if (*s < '0' || *s > '9') return false;
  return true;
}

void moveServo(const char *arg) {
  if (!isNumber(arg) || atoi(arg) > 180) {
    reply(F("Error: send a number 0 to 180"));
    return;
  }
  servoAngle = atoi(arg);
  servo.write(servoAngle);
  char out[32];
  snprintf(out, sizeof(out), "Servo moved to %d deg", servoAngle);
  reply(out);
}
```

```
void handleCommand(char *cmd) {
  cmd = trim(cmd);
  if (*cmd == '\0') return;
  char out[48];

  if (*cmd >= '0' && *cmd <= '9') {
    moveServo(cmd);                   // bare number, e.g. "90"
  } else if (strncasecmp(cmd, "SERVO:", 6) == 0) {
    moveServo(trim(cmd + 6));
  } else if (strcasecmp(cmd, "STATUS") == 0 || strcasecmp(cmd, "SERVO?") ==
0) {
    snprintf(out, sizeof(out), "Servo is at %d deg", servoAngle);
    reply(out);
  } else if (strcasecmp(cmd, "PING") == 0) {
    snprintf(out, sizeof(out), "%s here", NODE_NAME);
    reply(out);
  } else if (strcasecmp(cmd, "HELP") == 0) {
    reply(F("BT-SERVO commands:"));
    reply(F("0..180       move to angle"));
    reply(F("SERVO:<0-180> same, long form"));
    reply(F("STATUS       current angle"));
    reply(F("PING         node name"));
  } else {
    snprintf(out, sizeof(out), "Unknown: %s (send HELP)", cmd);
    reply(out);
  }
}

void pollStream(Stream &s, char *buf, uint8_t &len) {
  while (s.available()) {
    char c = s.read();
    if (c == '\n' || c == '\r') {
      if (len > 0) { buf[len] = '\0'; handleCommand(buf); len = 0; }
    } else if (len < CMD_MAX) {
      buf[len++] = c;
    }
  }
}

void setup() {
  Serial.begin(9600);
  bt.begin(9600);
```

```
  servo.attach(SERVO_PIN);
  servo.write(servoAngle);            // go to 90 deg on power-up
  reply(F("BT-SERVO ready at 90 deg. Send HELP."));
}

void loop() {
  pollStream(bt, btBuf, btLen);
  pollStream(Serial, usbBuf, usbLen);
}
```

How it works

- If a command starts with a digit it is taken as an angle; `SERVO:` does the same with a prefix.
- `isNumber()` accepts only 1–3 digits, and anything above 180 is refused, so a typo can never drive the servo past its end stop.
- `servo.write()` sets the angle, and the value is stored in `servoAngle`. `STATUS` reports this stored value; an SG90 has no position feedback, so the stored command is the best available answer.
- The servo moves to 90° at power-up so it always starts from a known position.

Test

Send `0`, then `180`, then `90`; the horn should sweep fully each way and return to centre. Send `STATUS` → `Servo is at 90 deg`.

Node 4 — Heartbeat (BT-HEART)

The node watches the pulse sensor all the time: `HR` asks for the current BPM, and `HRON` makes it message the phone on every beat it detects, without being asked.

Pins

This assumes a 3-pin analog pulse sensor (+, −, S), the common hobby type.

Sensor pin	UNO pin
+ (VCC)	5V
− (GND)	GND

Sensor pin	UNO pin
S (signal)	A0

The on-board LED (D13) flashes with each detected beat, so you can see detection working without the phone.

Commands

Send	What happens	Reply
`HR`	Reports the latest heart rate	`Heart rate: 74 BPM` or `No pulse yet - rest finger on sensor`
`HRON`	Sends a message on every beat	`Beat! BPM: 74` about once a second
`HROFF`	Stops beat messages	`Auto heartbeat: OFF`
`STATUS`	BPM, finger detected or not, auto mode	`HR: 74 BPM | finger: yes | auto: ON`
`PING` / `HELP`	Node name / command list	`BT-HEART here`

Code

No extra libraries needed; beat detection is written by hand.

In the repo: `nodes/04-heartbeat/BT_HEART/BT_HEART.ino`, with a node README in the same `nodes/04-heartbeat` folder.

```
/* Node 4 - BT-HEART : Arduino UNO + HC-05 + analog pulse sensor */
#include <SoftwareSerial.h>
#include <string.h>

const char NODE_NAME[] = "BT-HEART";
const uint8_t PULSE_PIN = A0;
const uint8_t BLINK_PIN = 13;       // on-board LED flashes on each beat

SoftwareSerial bt(2, 3);            // D2 <- HC-05 TXD, D3 -> HC-05 RXD
(divider)

const uint8_t CMD_MAX = 64;
```

```
char btBuf[CMD_MAX + 1];  uint8_t btLen = 0;
char usbBuf[CMD_MAX + 1]; uint8_t usbLen = 0;

// ---- detection settings ----
const unsigned long SAMPLE_MS  = 10;    // 100 samples per second
const int           MIN_SWING  = 40;    // smaller = no finger / just noise
const unsigned long MIN_IBI    = 300;   // faster than 200 BPM = noise
const unsigned long MAX_IBI    = 2000;  // slower than 30 BPM = noise
const unsigned long TIMEOUT_MS = 3000;  // no beat for 3 s = pulse lost

// ---- detection state ----
unsigned long lastSample = 0, lastBeat = 0;
int  peak = 512, trough = 512;
bool above = false, fingerOn = false, autoMode = false;
unsigned long ibi[5];
uint8_t ibiCount = 0, ibiIndex = 0;
int bpm = 0;

void reply(const char *m)                { bt.println(m); Serial.println(m);
}
void reply(const __FlashStringHelper *m) { bt.println(m); Serial.println(m);
}

char *trim(char *s) {
  while (*s == ' ') s++;
  size_t n = strlen(s);
  while (n > 0 && s[n - 1] == ' ') s[--n] = '\0';
  return s;
}

// ---- runs every loop, never blocks ----
void updateHeartbeat() {
  unsigned long now = millis();
  if (now - lastSample < SAMPLE_MS) return;
  lastSample = now;

  int v = analogRead(PULSE_PIN);

  // envelope: jumps to new highs/lows, then relaxes 1 step per sample
  peak   = (v > peak)   ? v : peak - 1;
  trough = (v < trough) ? v : trough + 1;
  int swing = peak - trough;
  int mid   = (peak + trough) / 2;
```

```
  int band  = swing / 8;              // hysteresis
  fingerOn  = swing >= MIN_SWING;

  if (!fingerOn) {
    above = false;
  } else if (!above && v > mid + band) {   // rising edge = beat
    above = true;
    digitalWrite(BLINK_PIN, HIGH);
    unsigned long gap = now - lastBeat;
    if (gap >= MIN_IBI) {
      lastBeat = now;
      if (gap <= MAX_IBI) {
        ibi[ibiIndex] = gap;
        ibiIndex = (ibiIndex + 1) % 5;
        if (ibiCount < 5) ibiCount++;
        unsigned long sum = 0;
        for (uint8_t i = 0; i < ibiCount; i++) sum += ibi[i];
        bpm = 60000UL / (sum / ibiCount);
        if (autoMode && ibiCount >= 2) {
          char out[24];
          snprintf(out, sizeof(out), "Beat! BPM: %d", bpm);
          reply(out);
        }
      }
    }
  } else if (above && v < mid - band) {    // signal fell back
    above = false;
    digitalWrite(BLINK_PIN, LOW);
  }

  if (bpm > 0 && now - lastBeat > TIMEOUT_MS) {
    bpm = 0; ibiCount = 0; ibiIndex = 0;
    digitalWrite(BLINK_PIN, LOW);
    if (autoMode) reply(F("Pulse lost - rest finger on sensor"));
  }
}

void handleCommand(char *cmd) {
  cmd = trim(cmd);
  if (*cmd == '\0') return;
  char out[56];

  if (strcasecmp(cmd, "HR") == 0) {
```

```
    if (bpm == 0) reply(F("No pulse yet - rest finger on sensor"));
    else {
      snprintf(out, sizeof(out), "Heart rate: %d BPM%s", bpm,
               ibiCount < 3 ? " (settling)" : "");
      reply(out);
    }
  } else if (strcasecmp(cmd, "HRON") == 0) {
    autoMode = true;
    reply(F("Auto heartbeat: ON"));
  } else if (strcasecmp(cmd, "HROFF") == 0) {
    autoMode = false;
    reply(F("Auto heartbeat: OFF"));
  } else if (strcasecmp(cmd, "STATUS") == 0) {
    snprintf(out, sizeof(out), "HR: %d BPM | finger: %s | auto: %s",
             bpm, fingerOn ? "yes" : "no", autoMode ? "ON" : "OFF");
    reply(out);
  } else if (strcasecmp(cmd, "PING") == 0) {
    snprintf(out, sizeof(out), "%s here", NODE_NAME);
    reply(out);
  } else if (strcasecmp(cmd, "HELP") == 0) {
    reply(F("BT-HEART commands:"));
    reply(F("HR          heart rate now"));
    reply(F("HRON        message on every beat"));
    reply(F("HROFF       stop beat messages"));
    reply(F("STATUS      BPM, finger, auto mode"));
    reply(F("PING        node name"));
  } else {
    snprintf(out, sizeof(out), "Unknown: %s (send HELP)", cmd);
    reply(out);
  }
}

void pollStream(Stream &s, char *buf, uint8_t &len) {
  while (s.available()) {
    char c = s.read();
    if (c == '\n' || c == '\r') {
      if (len > 0) { buf[len] = '\0'; handleCommand(buf); len = 0; }
    } else if (len < CMD_MAX) {
      buf[len++] = c;
    }
  }
}
```

```
void setup() {
  Serial.begin(9600);
  bt.begin(9600);
  pinMode(BLINK_PIN, OUTPUT);
  reply(F("BT-HEART ready. Send HELP."));
}

void loop() {
  pollStream(bt, btBuf, btLen);
  pollStream(Serial, usbBuf, usbLen);
  updateHeartbeat();
}
```

How it works

- `updateHeartbeat()` reads A0 every 10 ms. It tracks the signal's recent peak and trough; their midpoint is the beat threshold, so it adapts to each finger and the room light.
- Each rise above the threshold (plus a small hysteresis band) is one beat. The time since the previous beat is the inter-beat interval (IBI); gaps under 300 ms or over 2000 ms are discarded as noise.
- The last 5 good intervals are averaged, and BPM = 60000 / average IBI. `(settling)` is shown until 3 intervals are collected.
- If the swing between peak and trough is below 40, no finger is assumed; after 3 s with no beat the BPM resets to 0.
- This is a hobby sensor: finger pressure and movement change the reading, and it is not medically accurate.

Test

Rest a fingertip lightly on the sensor and keep still. Within 5–10 s the D13 LED should flash with your pulse; send `HR`, then `HRON` and watch beat messages arrive. Lift the finger → `Pulse lost` after 3 s.

Node 5 — LED (BT-LED)

Send `ON` or `OFF` and the LED follows; the node remembers the state so `STATUS` can report it.

Pins

LED leg	UNO pin
Anode (long leg)	D12 through a 220 Ω resistor
Cathode (short leg, flat side)	GND

Commands

Send	What happens	Reply
`ON` (or `LED:ON`, `1`)	LED on	`LED: ON`
`OFF` (or `LED:OFF`, `0`)	LED off	`LED: OFF`
`STATUS`	Reports the state	`LED is OFF`
`PING` / `HELP`	Node name / command list	`BT-LED here`

Code

No extra libraries needed.

In the repo: `nodes/05-led/BT_LED/BT_LED.ino`, with a node README in the same `nodes/05-led` folder.

```
/* Node 5 - BT-LED : Arduino UNO + HC-05 + LED */
#include <SoftwareSerial.h>
#include <string.h>

const char NODE_NAME[] = "BT-LED";
const uint8_t LED_PIN = 12;         // anode through 220 ohm

SoftwareSerial bt(2, 3);            // D2 <- HC-05 TXD, D3 -> HC-05 RXD
(divider)

const uint8_t CMD_MAX = 64;
char btBuf[CMD_MAX + 1];  uint8_t btLen = 0;
char usbBuf[CMD_MAX + 1]; uint8_t usbLen = 0;

bool ledOn = false;
```

```
void reply(const char *m)                { bt.println(m); Serial.println(m);
}
void reply(const __FlashStringHelper *m) { bt.println(m); Serial.println(m);
}

char *trim(char *s) {
  while (*s == ' ') s++;
  size_t n = strlen(s);
  while (n > 0 && s[n - 1] == ' ') s[--n] = '\0';
  return s;
}

void setLed(bool on) {
  ledOn = on;
  digitalWrite(LED_PIN, on ? HIGH : LOW);
  if (on) reply(F("LED: ON"));
  else    reply(F("LED: OFF"));
}

void handleCommand(char *cmd) {
  cmd = trim(cmd);
  if (*cmd == '\0') return;
  char out[48];

  if (strcasecmp(cmd, "ON") == 0 || strcasecmp(cmd, "LED:ON") == 0 ||
strcmp(cmd, "1") == 0) {
    setLed(true);
  } else if (strcasecmp(cmd, "OFF") == 0 || strcasecmp(cmd, "LED:OFF") == 0
|| strcmp(cmd, "0") == 0) {
    setLed(false);
  } else if (strcasecmp(cmd, "STATUS") == 0) {
    if (ledOn) reply(F("LED is ON"));
    else       reply(F("LED is OFF"));
  } else if (strcasecmp(cmd, "PING") == 0) {
    snprintf(out, sizeof(out), "%s here", NODE_NAME);
    reply(out);
  } else if (strcasecmp(cmd, "HELP") == 0) {
    reply(F("BT-LED commands:"));
    reply(F("ON  / LED:ON  / 1   LED on"));
    reply(F("OFF / LED:OFF / 0   LED off"));
    reply(F("STATUS              LED state"));
    reply(F("PING                node name"));
```

```
  } else {
    snprintf(out, sizeof(out), "Unknown: %s (send HELP)", cmd);
    reply(out);
  }
}

void pollStream(Stream &s, char *buf, uint8_t &len) {
  while (s.available()) {
    char c = s.read();
    if (c == '\n' || c == '\r') {
      if (len > 0) { buf[len] = '\0'; handleCommand(buf); len = 0; }
    } else if (len < CMD_MAX) {
      buf[len++] = c;
    }
  }
}

void setup() {
  Serial.begin(9600);
  bt.begin(9600);
  pinMode(LED_PIN, OUTPUT);
  digitalWrite(LED_PIN, LOW);         // start off
  reply(F("BT-LED ready. Send HELP."));
}

void loop() {
  pollStream(bt, btBuf, btLen);
  pollStream(Serial, usbBuf, usbLen);
}
```

How it works

- `handleCommand()` accepts three spellings for each state, so the short `ON` / `OFF` and app buttons sending `1` / `0` both work.
- `setLed()` drives D12 and stores the state in `ledOn`, which `STATUS` reads back.
- The 220 Ω resistor limits the current to roughly 13 mA, safe for both the LED and the UNO pin.

Test

Send `ON` → LED lights. Send `STATUS` → `LED is ON`. Send `0` → LED goes off.

Phone app: pairing and switching nodes

Pair all five nodes once; after that you control one node at a time by picking its name in the app's device list.

1. **Power all five nodes.** Each HC-05 LED blinks fast, which means it is waiting for a phone.
2. **Pair each one.** Phone Settings → Bluetooth → pair `BT-LCD`, `BT-DIST`, `BT-SERVO`, `BT-HEART` and `BT-LED`, each with PIN `1234` (or `0000`, or the PIN you set with `AT+PSWD`). On Android 12 or newer, also allow the app's "Nearby devices" permission.
3. **Install a terminal app** such as "Serial Bluetooth Terminal" from the Play Store.
4. **Set the line ending once.** Settings → Send → newline "CR+LF". Without it, no node sees the end of a command.
5. **Connect to a node.** Menu → Devices → Bluetooth Classic → tap the node name. The node's HC-05 LED switches to a slow double-blink when connected.
6. **Check which node you are on.** Send `PING`; the reply names the node.
7. **Switch nodes.** Disconnect (plug icon), go back to Devices, and tap another name. Switching takes a few seconds.
8. **Optional: macro buttons.** Long-press a macro button (M1, M2…) to save common commands, for example `DIST`, `ON`, `OFF`, `HR`, `90`, `STATUS`.

Two limits to know: an HC-05 accepts only one phone at a time, and most terminal apps hold one connection at a time. To see all five at once (for example heartbeat alerts while moving the servo), use a second phone or an app that supports several Bluetooth Classic connections.

Troubleshooting

Start with `PING`: if the right node answers, the Bluetooth link and base kit are fine, and the problem is in the component wiring.

Symptom	Node	Likely cause	Fix
Five devices all called "HC-05" on the phone	All	Modules not renamed	Follow "Naming each HC-05"; unpair and re-pair after renaming

Symptom	Node	Likely cause	Fix
`AT` gets no `OK`	All	Not in AT mode, or wrong baud	LED must blink slowly; pass-through sketch uses 38400 for the HC-05 and 9600 for the Serial Monitor
Connects but no reply	All	No line ending	Set CR+LF in the app
Garbled replies	All	HC-05 data speed not 9600	In AT mode send `AT+UART=9600,0,0`
Phone sends, node receives, phone gets nothing	All	Divider or D3 wiring	D3 → 1 kΩ → RXD, RXD → 2 kΩ → GND
Backlight on, no text	1 LCD	Wrong I2C address or contrast	Set `LCD_ADDR` to 0x3F; turn the blue contrast pot
Always `out of range`	2 Ultrasonic	Trig/Echo swapped, or soft/angled target	Trig → D7, Echo → D6; aim at a flat, hard surface
UNO resets when servo moves	3 Servo	Supply dips	Add the 470 µF capacitor; use a separate 5 V supply with shared GND
Servo twitches slightly while you type	3 Servo	SoftwareSerial briefly pauses interrupts	Normal and minor; it stops when no data is arriving
`No pulse yet` every time	4 Heartbeat	Finger moving or pressing hard, weak signal	Rest the finger lightly and still for 10 s; lower `MIN_SWING` to 20
BPM jumps or doubles	4 Heartbeat	Noise, room light	Shade the sensor; raise `MIN_SWING`
LED never lights	5 LED	LED reversed	Long leg toward the resistor and D12